

SHEPHERD

Unified declarative cross-platform fleet management system

Presented by: Radhey Kalra (2023A1R044) · Aabish Malik (2023A6R057)



Model Institute of Engineering and Technology (Autonomous), Jammu · 2026

The problem on campus

Institutions run computer labs with Linux, macOS, and Windows machines on one network, used by many students.

- Scripts and manual setup leave machines in different states. A lab of 30 machines drifts apart over a semester.
- Failed updates stop midway. Some machines boot, some do not, and nobody knows which without walking over.
- Student access adds constant untracked changes: settings, installs, deleted files.
- Current tools cover one OS each: SCCM and Intune for Windows, Jamf for macOS. Each needs a license and its own server. Updates pulled three times saturate the campus link.

What is Nix

Nix is a declarative package manager and configuration system. It allows reproducible, declarative management of software across multiple operating systems.

- **Store paths:** Every package builds into an isolated protected path. The hash covers all build inputs and ensures reproducibility.
- **No dependency conflicts:** Nix allows multiple versions of a library to coexist.
- **Generations:** Each configuration activation creates a numbered generation. Switching to the previous one is a symlink change.
- **Flakes:** flakes pin every input to an exact version, so two machines evaluate the same configuration into the same system.

Existing tools and what they miss

Tool	Coverage	Why it does not fit
SCCM / Intune	Windows	Per-seat licenses, policy model without state verification or rollback
Jamf	macOS	Separate license and server; no Linux or Windows
Ansible / shell scripts	Any	Push model: each run mutates host state; a failed run leaves the host half-configured
Deep Freeze	Lab disks	Restores disk on reboot but cannot install updates or change packages
NixOS alone	One machine	Declarative and rollback-capable, but no central dispatch, inventory, or non-Linux targets

Shepherd provides one declarative source for three operating systems, pull-based agents, signed artifacts, and rollback from the model rather than as an add-on.

How it works

A central server evaluates Nix Flakes into target states. Watchers on each machine enforce these states.

- **One source:** Administrators maintain a single Flake repository describing every host.
- **Pull model:** The server sends a target hash. Each machine fetches what it needs and applies it locally.
- **Verification:** Every archive carries a cryptographic signature. Watchers check signatures before unpacking.
- **Rollback:** A failed health check switches the host back to the previous generation.

Target users: campus system administrators, lab assistants, and institutional IT staff.

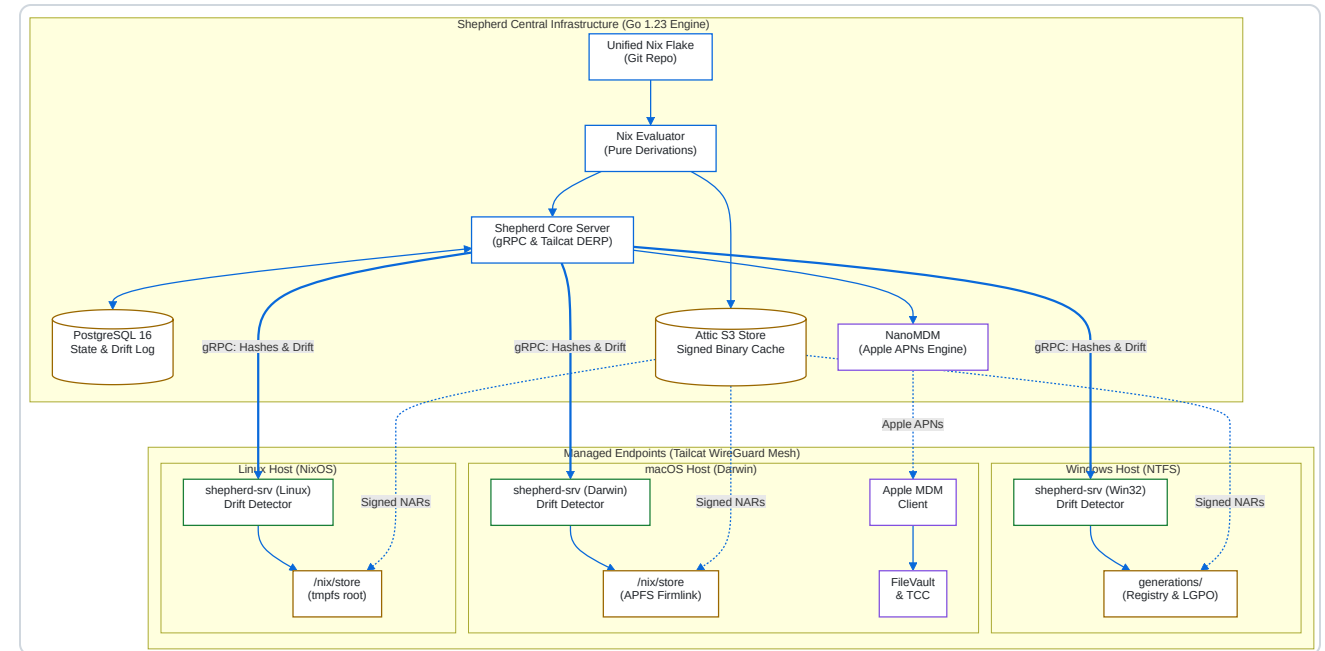
Onboarding a new machine

1. The technician boots the machine from an enrollment image (USB or PXE) containing `shepherd-srv` and host keys.
2. The client generates a device key pair and sends a join request.
3. An administrator approves the token in the web console.
4. The client receives its host name, role, and Flake target.
5. It pulls the signed closure from the cache or a LAN peer and verifies each signature.
6. It applies generation 1 and reports the resulting hash. The console lists the node as compliant.

After this, the node applies each new target hash on its own.

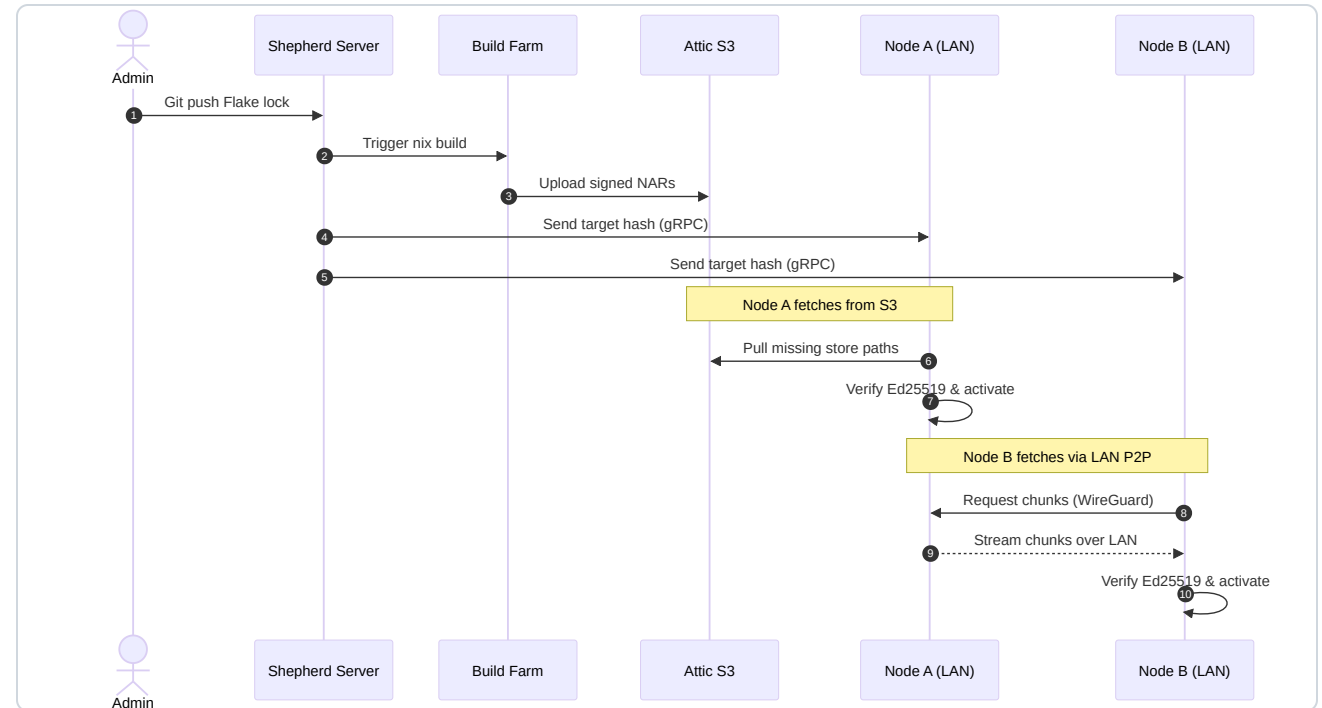
System architecture and topology

- **Shepherd server:** Evaluates unified Nix Flakes into target closures.
- **Control channel:** Outbound gRPC over HTTPS port 443 with mTLS.
- **Linux endpoints:** NixOS with shepherd client service.
- **macOS endpoints:** Nix-Darwin with embedded NanoMDM security.
- **Windows endpoints:** Native watcher service maps configurations to Win32, Registry, and LGPO.
- **P2P transport:** Tailcat userspace WireGuard mesh on local networks.



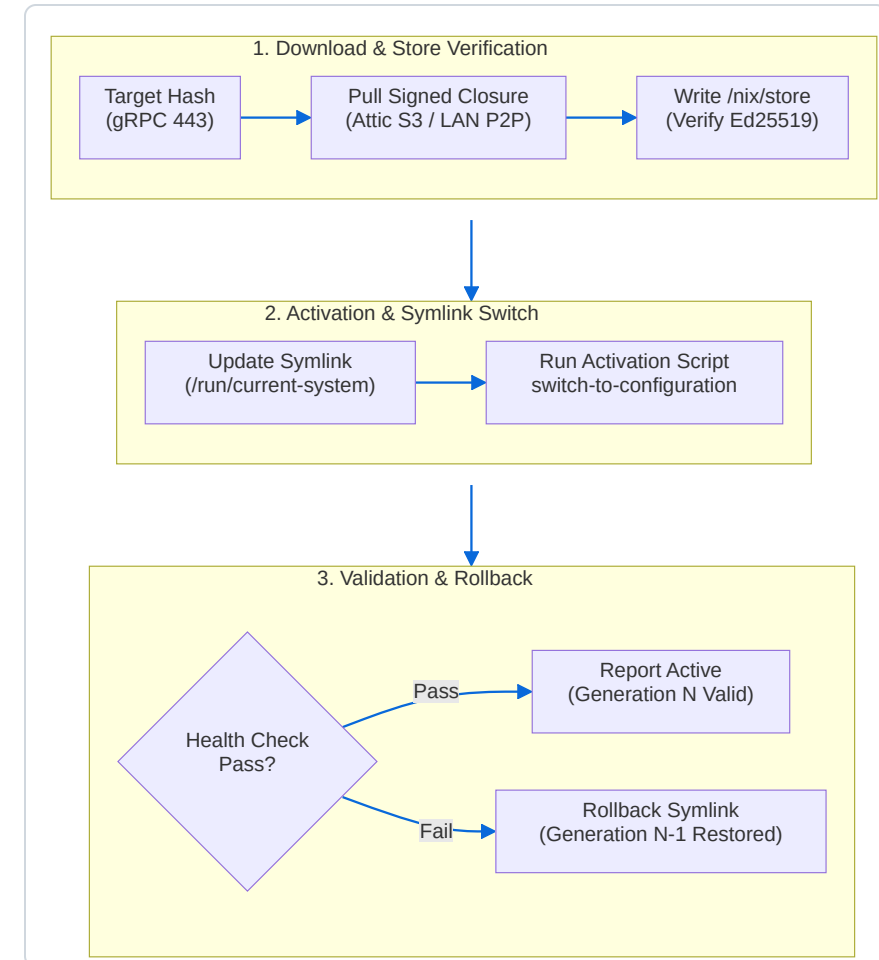
Build cache transfer sequence

- Admin pushes Flake update.
- Build workers compile derivations and write outputs to Attic.
- Server sends target hash to endpoints via gRPC.
- Node A downloads missing paths from Attic S3.
- Node B requests chunks from Node A over local LAN Tailcat WireGuard.
- Endpoints verify payload signatures before unpacking.



Linux node execution workflow

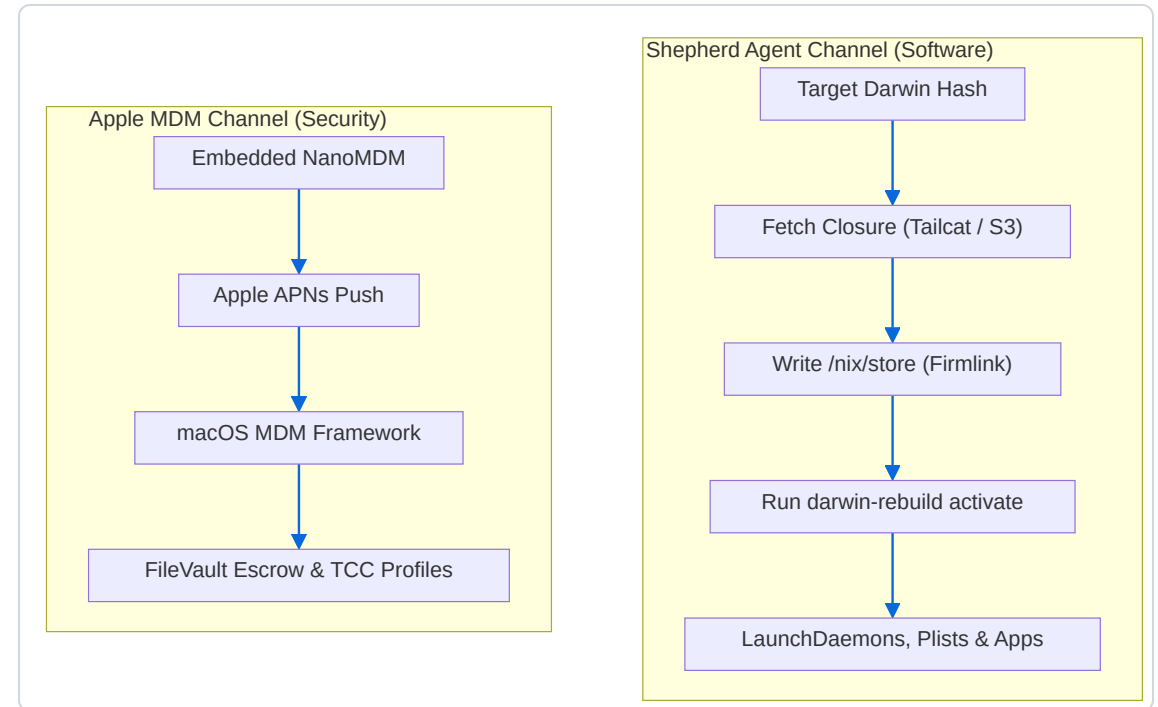
- **Ephemeral root:** The system boots with `tmpfs` mounted at `/`. A reboot clears all changes made after boot.
- **Store protection:** Read-only policy ensures non-administrator processes cannot modify system.
- **Rollback:** If post-switch health checks fail, the agent reverts the symlink to last safe generation.



macOS node execution workflow

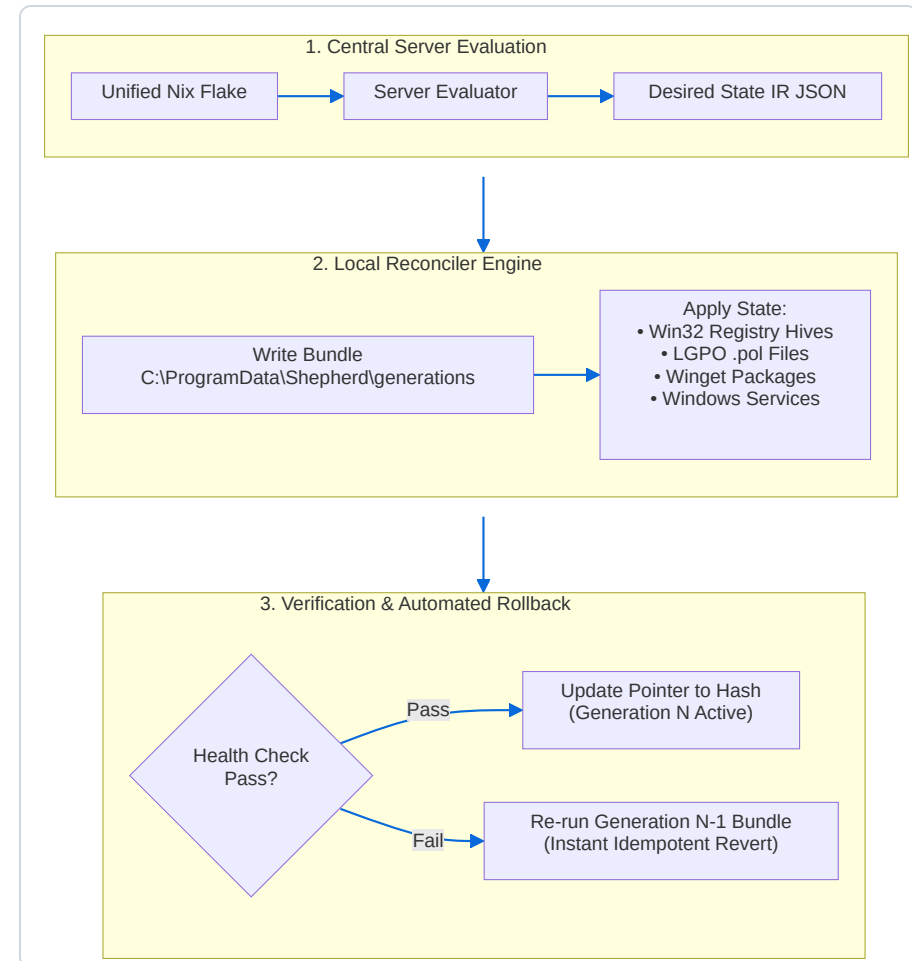
All macOS nodes require a trusted MDM through which the system can be configured and managed remotely.

- **MDM channel:** The server runs an embedded NanoMDM service for files and TCC (Transparency, Consent, and Control) management.
- **Nix channel:** Packages install in a read-only store through APFS synthetic firmlinks.
- **Licensing:** No third-party MDM subscription.



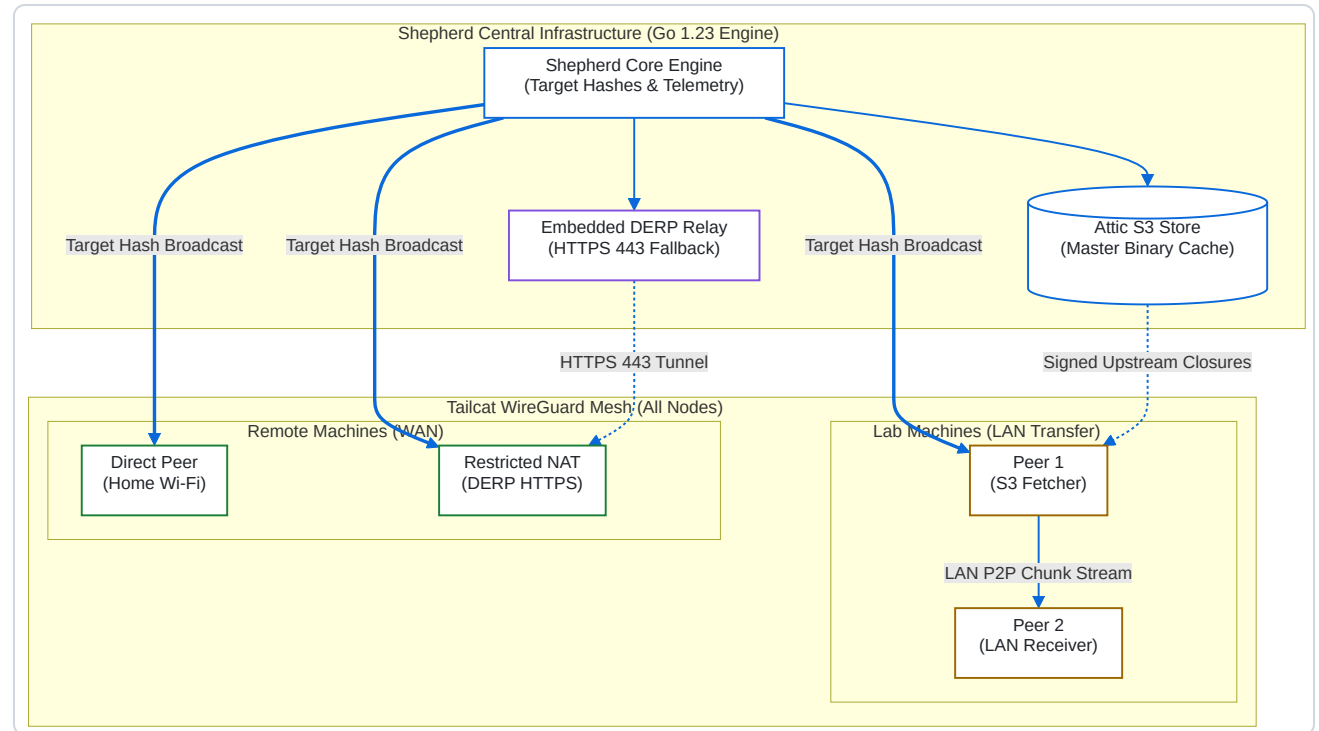
Windows node execution workflow

- **Intermediate representation:** The server evaluates Nix into a typed JSON desired-state document.
- **Generation bundle:** Client watcher writes files to a secure filepath.
- **State application:**
 - Writes registry keys through Win32 API calls.
 - Compiles and applies Group policy files.
 - Installs packages using the Windows Package Manager (WinGet).
 - Configures services and queries WMI.
- **Rollback:** If verification fails, the agent re-applies the last safe bundle.



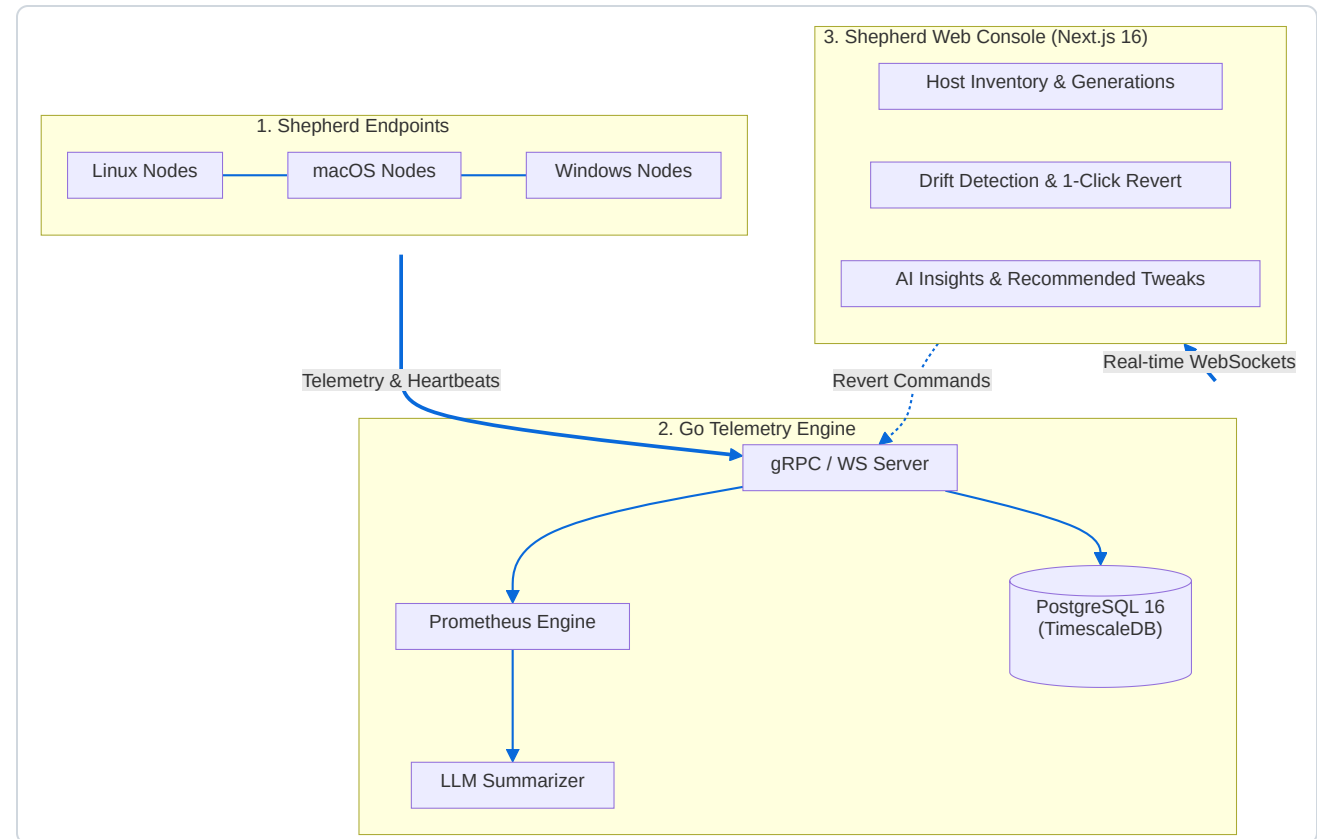
Networking: Tailcat mesh

- **Target hashes:** Desired-state hashes push over WireGuard. No open inbound ports.
- **Telemetry:** Heartbeats and metrics stream via gRPC, with HTTPS 443 DERP fallback.
- **Remote machines:** Userspace WireGuard (gVisor `netstack`, no TUN conflicts) with DERP relay behind NAT.
- **Lab machines:** Local peers discover via Tailcat Disco and swarm store chunks at LAN speed.



Web console and telemetry

- **Web console:** Live node inventory, NoCode target generations, and drift state.
- **Telemetry stream:** Ingests node metrics and heartbeats via Go gRPC into PostgreSQL.
- **AI incident insights:** Explains Prometheus alerts and metric spikes with plain-language root-cause notes.
- **AI-recommended tweaks:** 1-click hardware-matched Flake tweaks and configuration diffs for admin approval.
- **1-click revert:** Administrator confirms AI proposals and triggers instant rollback for drifting nodes.



Daily operation

- **Drift reports:** Agents report registry edits, changed services, and untracked packages. The console pairs each report with the assigned generation.
- **Revert:** The administrator selects non-compliant nodes and applies the assigned generation again. Bulk actions cover a whole lab at once.
- **AI-drafted packages:** When course software is missing from repositories, a model drafts the Nix derivation. It deploys only after the same sandboxed build, hash, and reproducibility checks as any other package.
- **Hardware-matched settings:** The model reads CPU, RAM, disk, and GPU reports and proposes Flake changes per machine class (effects off on low-RAM lab desktops, build parallelism matched to core count). The administrator accepts or edits each proposal as a normal Flake diff.
- **Limits:** The assistant produces drafts and summaries. It does not change running machines on its own; every change passes through the same review and generation pipeline as manual work.

Technology stack

Subsystem	Technology	Role
Control plane	Go 1.23	HTTP/gRPC server, Nix evaluator, embedded Tailcat DERP relay
Apple MDM	NanoMDM	APNs push, FileVault key escrow, TCC profiles
Database	PostgreSQL 16	Node inventory, optional TimescaleDB telemetry
Build cache	Attic + S3	Content-addressed store, FastCDC deduplication
Client agent	Go 1.23	systemd / launchd / Windows service
Windows engine	Win32 FFI	Registry writes, LGPO policies, Winget installs
Web console	Next.js 16	React 19, TypeScript, Tailwind CSS
Analysis	LLM pipeline	Log summaries, derivation drafts, hardware tuning proposals
Networking	WireGuard + Tailcat	peer-to-peer communication, remote access

V.E.T.S justification

Criterion	How the project meets it
V: Viability	Fits one semester. Built on Go 1.23, Nix, WireGuard, PostgreSQL. A three-OS lab testbed is running.
E: Engineering depth	Nix-to-Win32/LGPO/plist translation, Ed25519 closure verification, userspace WireGuard with NAT traversal, rollback under three seconds.
T: Trend alignment	Infrastructure as code, immutable OS roots, Zero Trust with outbound-only mTLS, applied models for operations.
S: Social impact	No SCCM/Intune/Jamf licenses; LAN chunk swarming keeps large rollouts off the campus WAN; labs return to a clean state on reboot.

Expected outcomes

- **Central server** ([shepherd](#)): Go 1.23, gRPC services, embedded DERP relay, audit logging.
- **Endpoint agents** ([shepherd-srv](#)): native clients for Linux, macOS, and Windows.
- **Flake template**: host modules for NixOS, nix-darwin, and the Windows generation engine.
- **P2P distribution**: verified closure transfer between LAN peers.
- **Drift handling**: detection and rollback within three seconds.
- **Analysis pipeline**: log summaries, derivation drafting with validation, hardware-tuned proposals.
- **Benchmarks**: convergence time, WAN traffic saved, rollback reliability.

Implementation timeline

Phase	Weeks	Activity
Phase 1	1–3	Requirement analysis, schema design, multi-OS testbed setup.
Phase 2	4–7	<code>shepherd</code> server in Go 1.23, PostgreSQL 16 schema, Flake evaluator.
Phase 3	8–11	<code>shepherd-srv</code> agents: Linux tmpfs root, macOS APFS/MDM, Windows Win32/LGPO.
Phase 4	12–14	Tailcat WireGuard mesh, DERP relay, FastCDC LAN transfer.
Phase 5	15–16	Integration testing, drift evaluation, benchmarks, synopsis presentation.

Summary

- One Flake repository defines Linux, macOS, and Windows machines.
- Agents pull signed target hashes and verify each artifact before activation.
- Failed updates roll back to the previous generation; Linux labs also reset on reboot.
- LAN peers exchange store chunks, so campus bandwidth stays free during rollouts.
- A model drafts derivations and summarizes logs; administrators review every change.